

课程笔记

静下心来，读懂 DeepSeek V4 的底层逻辑

原讲者：梗直哥、 笔记整理：Claude Code

2026 年 4 月 27 日



视频作者/频道：梗直哥、（Bilibili）

发布日期：2026-04-27

视频时长：22 分 20 秒

视频链接：<https://www.bilibili.com/video/BV179oeBeEcN/>

目录

1 记忆：长短期多分辨率动态记忆与 CSA	3
1.1 从「图谋」说起：V4 真正改了什么	3
1.2 CSA 的四件套：从「框图一摆」到看懂本质	4
1.3 为什么说 CSA 是「Transformer 版的 LSTM」	5
1.4 MLA「单层静态」vs CSA「多分辨率动态」	6
1.5 结果只是表象：从「百万上下文」回到「记忆本质」	6
1.6 本章小结	7
1.7 拓展阅读	7
2 遗忘：主动遗忘机制与 HCA	8
2.1 被「朴实名字」骗住的第一直觉	8
2.2 反常识的神经生物学：调控遗忘反而巩固记忆	9
2.3 HCA = 主动遗忘：怎么「忘」才不丢主干	9
2.4 为什么这一对必须分两支，不能合一支	10
2.5 本章小结	10
2.6 拓展阅读	10
3 节律：CSA / HCA 交错的睡眠—觉醒结构	12
3.1 动机：为什么不能“既要又要”	12
3.2 主图：均匀交错的层间排布	12
3.3 排除法：四种“看起来也行”的方案为什么都不行	13
3.4 机制：把节律设计写进 Transformer	14
3.5 本章小结	14
4 内生 RAG：把检索焊进 Attention（含 MoE 与 MHC 工程优化）	15
4.1 V4 attention block：双路计算单元，不是旁路	15
4.2 从外挂 RAG 到原生检索增强	16
4.3 MoE：决策层只做工程优化	17
4.4 MHC：传播层的智能阀门	19
4.5 三层归纳：V4 改动的总体地图	19
4.6 本章小结	20
5 内生 Agent 化：把 Agent 做进模型里	22
5.1 为什么 DeepSeek 一年半不做 Agent？	22
5.2 老式 input-output（图 B）vs V4 内部自循环（图 A）	22
5.3 Request 级别的调用模型：SWA + KV state	23
5.4 从外挂记忆到内生记忆：梁文峰看到的「Agent 死穴」	25
5.5 重要边界：内生 Agent 化 $\neq$ 完整 Agent	25
5.6 本章小结	25

6 总结与延伸	26
6.1 五要点闭环	26
6.2 三层归纳再确认	27
6.3 弱 Agent 危机：哪些 Agent 会消亡，哪些不会	27
6.4 Router 趋势：从手动模式切换到 automatic routing	28
6.5 后训练 / 行业能力被吃进通用大模型	28
6.6 终局判断：从「模型 + Agent」到「模型即 Agent」	29
6.7 笔者综合：跨章节压缩与对学习者的实操含义	29
6.8 本章小结	30
6.9 拓展阅读	30

1 记忆：长短期多分辨率动态记忆与 CSA

1.1 从「图谋」说起：V4 真正改了什么

DeepSeek V4 发布的那句话，多数人「只重其表意」——把它看成又一次架构小调、跑分小升。讲者梗直哥的判断恰好相反：DeepSeek 憋了一年半放出的这一招，背后是一次直指「记忆」本质的底层革命。所谓「支持百万 token 超长上下文、降算力、降显存」的口号，只是结果和表现，不是底层逻辑。

把 V4 与 V3 的整体结构平铺开来对比（图 1），我们立即看到：

- 上方的多 Token 预测（Multi-Token Prediction, MTP）模块——原样照搬；
- 下方的混合专家（Mixture of Experts, MoE）模块——整体保留，仅作工程级局部优化（留待 §4 细谈）；
- 中间那一块由多头潜变量注意力（Multi-head Latent Attention, MLA）担当的 attention 子层——被整体替换为两个并列模块：压缩稀疏注意力（Compressed Sparse Attention, CSA）与重度压缩注意力（Heavy Compression Attention, HCA）。

换言之，V4 的「真改」并不在 MTP 或 MoE，而死死地集中在 attention 子层。要读懂 V4 的「图谋」，就必须先读懂 CSA 与 HCA。本章先讲 CSA——主动记忆机制；下一章讲 HCA——主动遗忘机制。

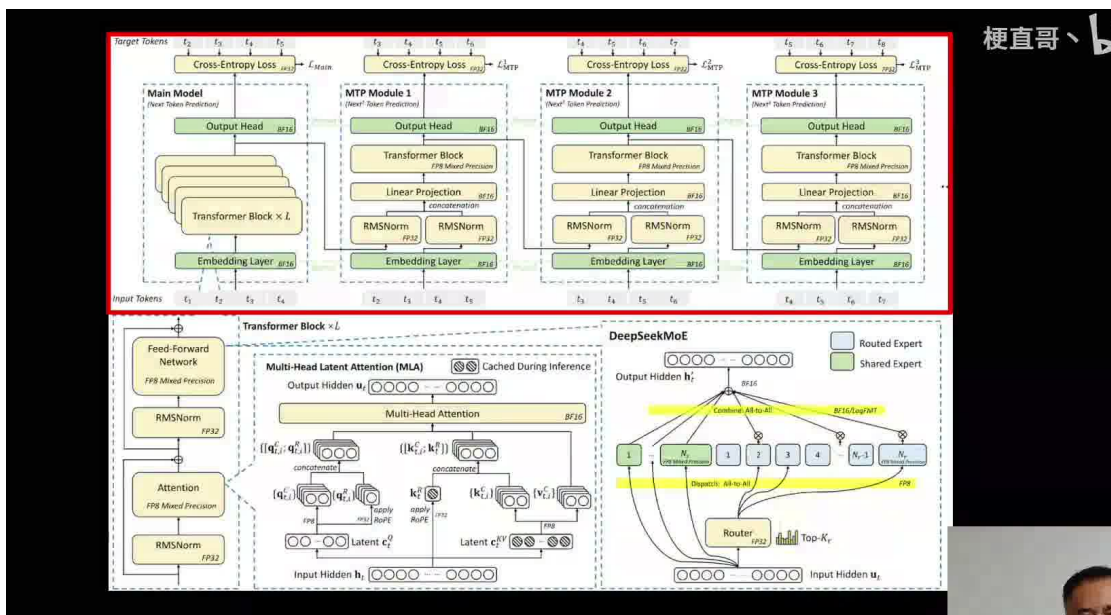


图 1: V3 与 V4 的整体结构对照：MTP 原样保留，MoE 整体保留并局部优化，唯独 attention 子层由 MLA 整体替换为 CSA + HCA¹

¹ 视频画面时间区间：00:01:09–00:01:30。

[

title= 前置回顾：V3 的 MLA 在做什么] MLA 把所有过往 token 的 KV (key-value) 缓存压成一组共享的潜变量 (latent)，一次性、全局、静态地完成压缩——压完之后整层注意力就只看这组潜变量。它解决的是显存占用问题，但代价是：所有 query 都看到同一份「平均化」的过去，无法按当前问题动态地拉细节。这正是 V4 想要换掉它的根本原因。

1.2 CSA 的四件套：从「框图一摆」到看懂本质

动机 单看 CSA 的全名「压缩稀疏注意力」拗口，单看官方框图也不易抓住骨架。讲者给出的破题口诀是一句：

CSA 是一个 *Transformer* 版的长短期记忆机制 (*LSTM*)。

这句话是本章最重要的一行。请把它先放在心里，再看下面四件套如何把这件事做成 (图 2)。

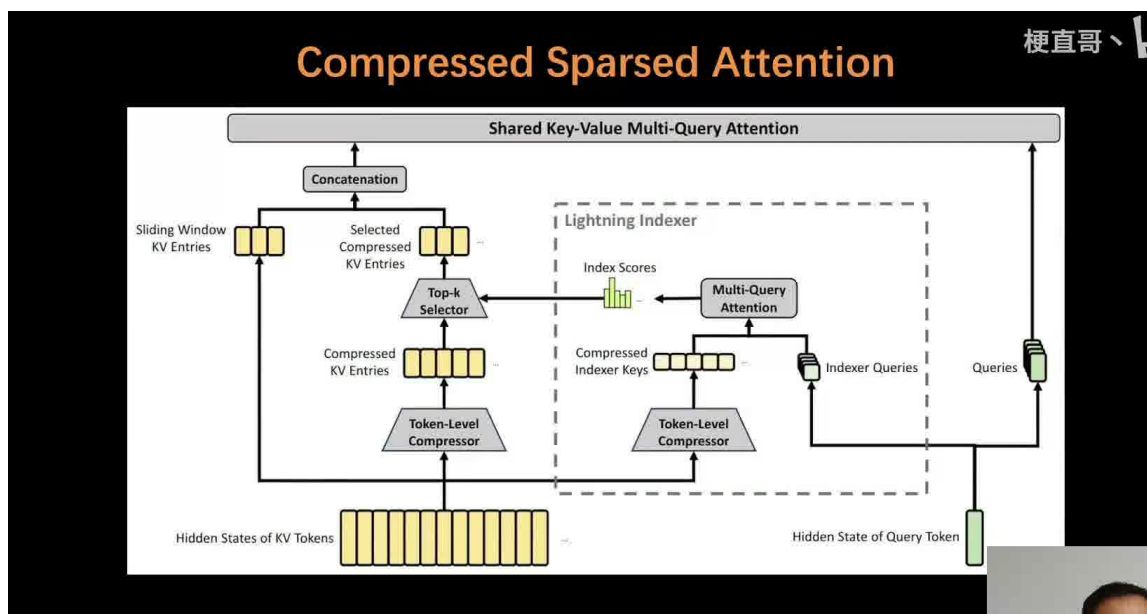


图 2: CSA 的四件套：底部黄色长条是全量 KV 记忆池（可达数十万 token），左侧 sliding window 保短期，token-level compressor 把全量压缩成可索引的池子，indexer 由 query 派生出 index queries 与 index scores，最终用 multi-query attention 完成 top-k 召回²

机制（顺时针看图）

1. **全量 KV 记忆池**（图底部黄色长条）：保留几十万 token 的原始 K/V，作为「长期、全量、未压缩」的底层池子。它不会直接参与每一次 attention 计算，否则显存会爆。
2. **滑动窗口（Sliding Window Attention, SWA）**：左侧那条窄窄的窗口只保最近若干千个 token 的 K/V，做短期、不压缩的精细 attention——「就像翻一本书时盯着的最近两三页」。
3. **Token 级压缩器（token-level compressor）**：把底部的全量 KV 做一次高倍压缩，得到一个可被快速检索的小型记忆池——这是「长期记忆」的可访问形式。

²视频画面时间区间：00:01:30-00:03:00。

4. 索引器 (indexer) + 索引查询 (index queries) + 索引分数 (index scores): 当前 query 不是直接去和压缩 KV 算 attention, 而是先派生一组小型 index queries, 对压缩池打分, 得到 index scores; 用这组分数选出最相关的 **top-k** 条 KV, 再把它们送进 **多查询注意力 (Multi-Query Attention, MQA)** 完成实际的内容读取。

讲者的大白话很到位:「相当于用 query 这边的蛛丝马迹, 去长序列里面检索我感兴趣的记忆。」短期由滑动窗口兜底, 长期由「压缩池 + indexer + top-k」按需召回——**动态、稀疏、查询驱动**。这就是「Compressed Sparse Attention」名字的字面来源。

1.3 为什么说 CSA 是「Transformer 版的 LSTM」

类比的精确含义 CSA 与 长短期记忆网络 (Long Short-Term Memory, LSTM) 在哲学上同构, 但实现上完全不同。表 1 把这种「同构而不同形」摊开来看。

功能维度	LSTM 的做法	CSA 的做法
短期记忆	cell state 中近期累积部分	滑动窗口 SWA
长期记忆	cell state + forget gate 累计	压缩 KV 池 + indexer
按需取用	output gate 控制读出	index scores 做 top-k 召回
显式门控	三道 gate (input/forget/output)	无额外 gating , 复用 attention
是否另起炉灶	全新一套 RNN cell	不另起炉灶, 留在 Transformer 内

表 1: CSA 与 LSTM 的哲学等价对照

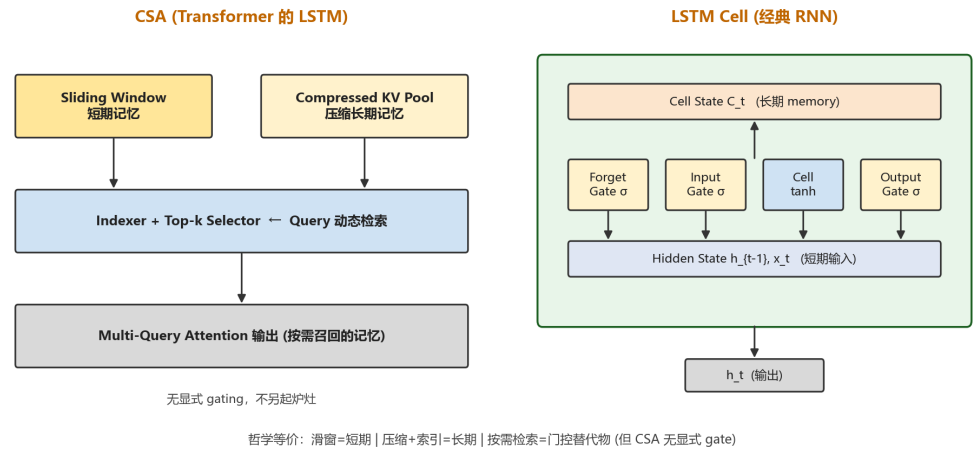


图 3: CSA 与 LSTM 的哲学等价示意: 左侧 CSA 的滑窗 + 压缩池 + indexer 三块对位 LSTM 右侧的 cell state 与门控; CSA 没有引入显式 gate, 亦未另起炉灶³

讲者的工艺评语 讲者把这种克制称为「螺壳里做道场」——不引入额外门控、不发明新结构, 就在 Transformer 既有的 Q/K/V/attention 机制里把短期与长期、稠密与稀疏、保留与压缩四件事一次理顺。这是**极客 + 匠人**的做派, 是在浮躁 AI 氛围下值得反复学习的「定力与长期主义」。

³ 视频画面时间区间: 00:02:43–00:03:25。

[

title= 本节最重要的一行] **CSA = Transformer 版的 LSTM**：滑窗保短期，压缩 KV 池 + indexer 保长期，top-k MQA 实现按需检索，全程不另立门控、不另起炉灶。理解了这一点，再看 CSA 框图就不再觉得拗口。

1.4 MLA「单层静态」vs CSA「多分辨率动态」

从静态到动态 V3 的 MLA 是单层、一次性、静态的全局压缩——所有 query 都从同一份压缩潜变量里取信息，无论你这次问的是什么、需要细节还是主旨，看到的过去都是同一份「平均」。CSA 则不同：每个 query 都会独立地通过 indexer 去全量压缩池里挑它自己关心的那 top-k 条 KV。

多分辨率金字塔 这种「query-driven 动态筛选」最重要的副产品，是一种事实上的多分辨率金字塔（图 4）：浅查询拿粗粒度主干，深查询能下钻到原始细节，分辨率不再被一刀切死，而是随当前 query 动态浮动。

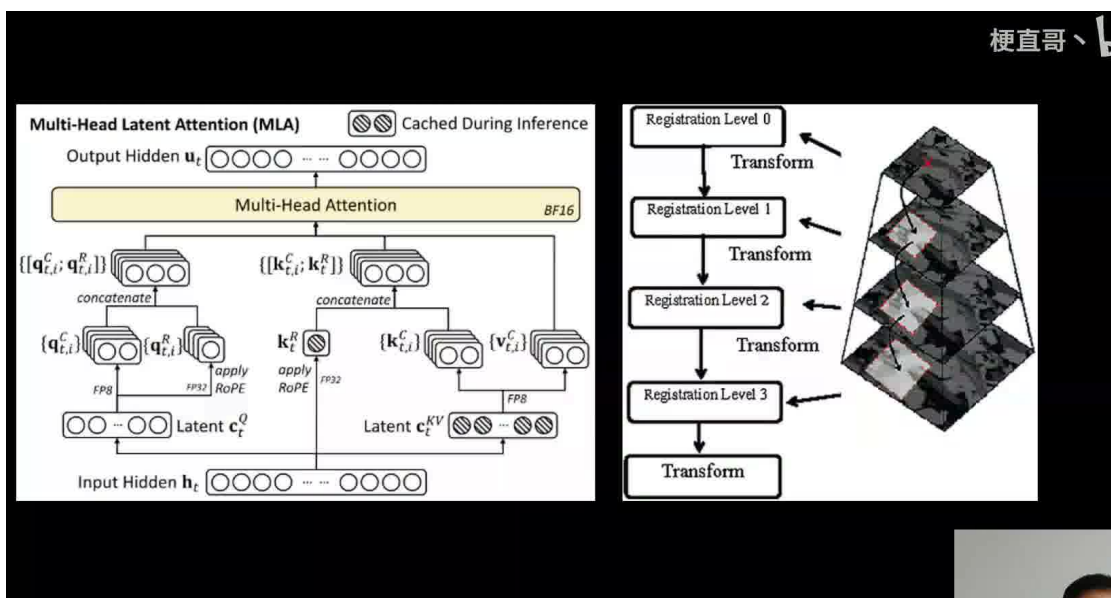


图 4: MLA 单层静态压缩（左）vs CSA 多分辨率动态金字塔（右）：MLA 把过去拍扁成一份「平均潜变量」，CSA 让每个 query 在金字塔不同层级动态切片⁴

[

title= 容易搞反的认知] 不要把 CSA 当成「MLA 的多层堆叠」或「MLA 加了个滑窗」。两者的关键差别不在层数、不在窗口，而在**压缩是否随 query 动态化**：MLA 是静态压缩 + 全员共享，CSA 是压缩仍存 + query 驱动 top-k 召回。前者是定格快照，后者是按需点播。

1.5 结果只是表象：从「百万上下文」回到「记忆本质」

至此可以回答开头那个问题：为什么我们说百万 token 上下文、降算力、降显存只是结果？

⁴视频画面时间区间：00:03:25–00:04:05。

因为它们只是 CSA 「短期 + 长期 + 按需召回 + 不另起炉灶」这套**记忆架构**所附带的工程性副产品。把因果反过来看才是底层逻辑：DeepSeek 是先想清楚「LLM 真正缺的是什么样的记忆」，再设计出 CSA，最后才顺势收下「百万上下文」「省显存」这些福利。这也是讲者说的——**V4 的真正图谋**是直指「记忆的本质」。

1.6 本章小结

[

title=§1 一图记忆]

- **V4 的真改在 attention**：MTP 照搬、MoE 工程优化，唯独 MLA 被整体替换为 CSA + HCA。
- **CSA 四件套**：滑动窗口（短期）+ token 级压缩器（长期可索引化）+ indexer/index scores（动态打分）+ top-k 多查询注意力（按需召回）。
- **哲学等价**：CSA 是「Transformer 版的 LSTM」——无显式门控、不另起炉灶，在 Transformer 内部把长短期记忆做出来。
- **对比 MLA**：MLA 单层静态全局压缩；CSA query 驱动、动态金字塔、多分辨率。
- **底层逻辑**：百万 token 上下文与降显存只是结果，CSA 真正瞄准的是记忆的本质。

1.7 拓展阅读

- LSTM 的原始动机与门控直觉：S. Hochreiter & J. Schmidhuber, *Long Short-Term Memory*, 1997——读完后再看 CSA，会更清楚「短期/长期/按需」三件事在不同范式下的对位。
- DeepSeek V3 与 MLA 技术报告：用以对照「单层静态压缩」的具体形式，是理解 V4 「为什么必须换掉 MLA」的最短路径。

2 遗忘：主动遗忘机制与 HCA

2.1 被「朴实名字」骗住的第一直觉

动机 上一章我们看清了 CSA (§1 介绍的主动记忆模块)：滑动窗口保短期、压缩 KV 池 + indexer + top-k 召回保长期。本章轮到 V4 attention 子层的另一支——**重度压缩注意力(Heavy Compression Attention, HCA)**——出场 (§1 已点过名，本章正式展开)。几乎所有人对 HCA 都会有一个看似合理的第一反应：

HCA 不就是把 CSA 的右半边——基于 *query* 的动态检索召回——砍掉，留下滑窗加一个压缩得更狠的 KV 通路嘛？听上去就是 CSA 的简化版。

讲者坦言自己一开始也是这么以为的，但越想越不对劲：如果 HCA 只是「压得更狠」，那何必单立一支？只要在 CSA 里把压缩比调高即可。单立一支，必有不可替代的角色。

结构上的差别 从图 5 可以看到，HCA 在结构上确实剪掉了 CSA 中那条 *indexer* → *index queries* → *index scores* → *top-k MQA* 的动态检索通路（图中灰色虚线部分），保留下来的只是「滑动窗口 + 超高倍 token-level compressor → 直接进入 attention」。这是「砍掉了哪些」，但还没回答「为什么要这样砍」。

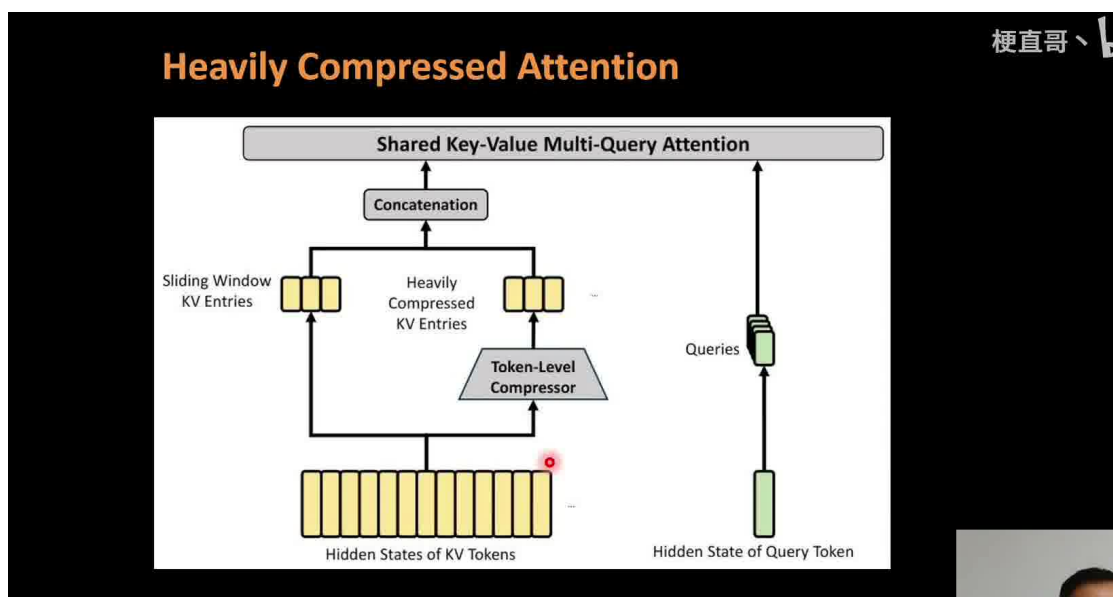


图 5: HCA 与 CSA 同形对照：基于 *query* 的动态检索召回 (*indexer* / *index scores* / *top-k MQA*) 整支被剪除，只留下滑窗 + 超高倍 KV 压缩器；这不是「压得更狠的 CSA」，而是另一种机制⁵

[

title= 务必避免的误读] HCA 不是 CSA 的简化版。如果只是「同一种机制 + 更高压缩比」，CSA 一行超参就能做到，不需要新模块。HCA 的存在意味着：它在做另一件事——一种与「按需检索回忆」性质上不同的、对记忆的主动处理。

⁵ 视频画面时间区间：00:04:05–00:04:50。

2.2 反常识的神经生物学：调控遗忘反而巩固记忆

常识的反面 我们日常把「遗忘」视作一种缺陷：忘了重要的事、忘了名字、忘了细节。但近年神经生物学给出了另一面证据：适度且主动的遗忘，反而能巩固记忆。

[

title=Scripps 2022 / PNAS: 主动遗忘巩固记忆] 2022 年，美国斯克里普斯研究所（Scripps Research Institute）在顶刊 *PNAS*（Proceedings of the National Academy of Sciences）发表的研究指出：通过**调控遗忘**（active forgetting）反而可以巩固记忆。机制并不玄：当大量记忆累积，彼此之间会形成**相互干扰**（interference）——不一致或不相关的细节噪声会拖累主干。主动地清除这些干扰项，留下的主干信号反而更稳。

从生物到模型 把这个发现搬到 LLM 上，逻辑就通了：一个能塞进百万 token 的模型，如果对所有过往细节一视同仁地保留，反而会被海量旧细节里的噪声、重复、冲突拖垮。它需要一种机制，主动地把「干扰多于信号」的内容抹去——不是当作召回候选，而是直接从注意力可见的表征里压成一片粗粒度主干（图 6）。

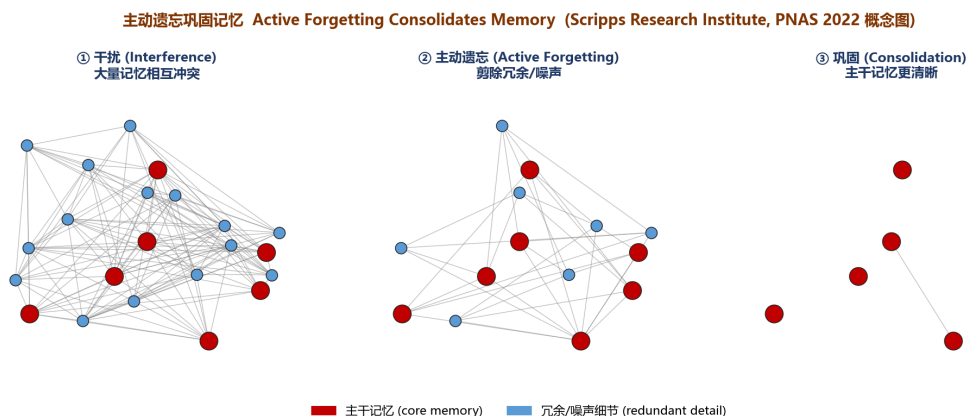


图 6: 主动遗忘巩固记忆的概念示意（基于 Scripps 2022 / PNAS）：左 = 大量细节相互干扰；中 = 主动遗忘剪除冗余与冲突；右 = 主干被提纯，整体记忆质量提升⁶

2.3 HCA = 主动遗忘：怎么「忘」才不丢主干

机制 HCA 走的就是这条主动遗忘路线，但有一个关键克制：

- 不硬删除任何 token。HCA 不会直接把过去的 KV 从序列里抠掉——一旦硬删，丢失就是不可逆的。
- 用超高倍压缩做「软抹除」。它对全量 KV 做远超 CSA 的强压缩，让大量细节、噪声、冗余在压缩过程中被自然抹平，只剩下粗粒度的主干信号。
- 不再做基于 query 的动态检索。因为这一支的目的不是「按需读细节」（那是 CSA 的活），而是「把噪声从输入端就拦下来、不让它污染后续注意力」。

⁶视频解说时间区间：00:04:50–00:05:35。

讲者的总结很直接——HCA 实现的是「细节过滤、噪声去除、主干提纯」，从而避免模型被海量旧细节拖垮。

[

title=CSA 与 HCA 的真正分工]

- **CSA = 主动记忆**：动态、稀疏、查询驱动地把需要的细节从全量池里召回。
- **HCA = 主动遗忘**：通过超高倍压缩在表征层把不需要的细节过滤掉，只保留主干。
- **二者互补、不冗余**：一个负责记得起来，一个负责不被记住的东西拖累。把它们合起来，才是一个完整的「记忆系统」。

2.4 为什么这一对必须分两支，不能合一支

回到本章最初那个反问：「能不能在 CSA 里把压缩比调到极高，一支搞定？」答案是不能，原因有两条：

1. **目标函数不同**。CSA 想保留可恢复的细节以待 query 召回，所以压缩比受限；HCA 想丢掉干扰细节，必须压得很狠。让一支同时担两职，超参一定打架。
2. **下游消费方式不同**。CSA 的产物要进 top-k MQA 做选择性消费；HCA 的产物则直接作为粗粒度上下文进 attention 主干。前者是「按需翻字典」，后者是「读个总结开头」——结构上注定要拆。

这也为下一章「节律：CSA 与 HCA 在层间如何交错」埋下伏笔——既然两支机制功能互补，那么把它们怎么排才能让模型「白天记细节、夜晚提纯主干」？这正是 §3 的主题。

2.5 本章小结

[

title=§2 一图记忆]

- **HCA 不是 CSA 的简化版**：若只是压得更狠，调 CSA 超参即可，无需另立一支。
- **神经生物学依据**：Scripps 2022 (PNAS) 证明主动遗忘能通过消除干扰来巩固主干记忆。
- **HCA 的本质**：用超高倍压缩做「软抹除」——不硬删 token，但把细节、噪声、冗余压成粗粒度主干，避免模型被旧细节拖垮。
- **CSA + HCA 的分工**：CSA 主动记，HCA 主动忘；前者按 query 召回需要的细节，后者从源头拦截噪声。互补而非冗余。

2.6 拓展阅读

- Scripps Research Institute, *Active forgetting consolidates memory*, PNAS, 2022——HCA 设计哲学的神经生物学锚点；阅读时重点抓「干扰 → 主动遗忘 → 巩固」三段式。

- 经典综述 R. L. Davis & Y. Zhong, *The Biology of Forgetting* (Neuron) ——把「遗忘是机制不是缺陷」这件事讲清楚的一篇背景读物，对理解 HCA 的「主动」二字尤其有帮助。

3 节律：CSA / HCA 交错的睡眠—觉醒结构

我们已经在 §1 与 §2 中分别看到了 V4 attention 子层的两块拼图：CSA 负责主动记忆、HCA 负责主动遗忘。两者就像一组互补的“开关”，一个抓细节、一个提主干。但仅有这两个模块还不够——怎么把它们装回 *Transformer*，才能让模型既记得住、又忘得掉、还训得稳？本节聚焦讲者的第三个要点：节律（circadian rhythm / sleep）。

3.1 动机：为什么不能“既要又要”

把 CSA 和 HCA 装进同一个 *Transformer* 的方案乍看有很多种：全用 CSA、全用 HCA、同一层并行、前 CSA 后 HCA……每一种听起来都可以“既要又要”。但讲者提醒读者先问一个更朴素的问题：人脑是怎么处理记忆的？

读者多半都有过这种经验：临睡前学一段东西，第二天起来反倒记得更牢。这并不是错觉，神经生物学顶刊⁷多年前就有针对睡眠—记忆固化的研究：醒着的时候大脑负责编码细节，睡着的时候负责压缩冗余、提纯主干。这恰好对上了我们手里的两个模块——

- **CSA**：白天觉醒，记细节、做检索；
- **HCA**：夜晚睡眠，压冗余、提纯主干。

如果模型能照着这个生物节律来排布两种 attention，“干一会儿、睡一会儿”，理论上就能越学越稳。这就是 V4 选择 **layer-wise 均匀交错** 的原始动机。

3.2 主图：均匀交错的层间排布

把目光放回结构图，可以看到 V4 在 *Transformer* 的 layer 维度上，把 CSA 和 HCA 一层一层地交替堆叠（图 7）：绿色 CSA 与黄色 HCA 像白昼与黑夜一样轮替出现。这并不是一个工程上的随手处理，而是讲者要重点强调的“机制选择”。

⁷讲者口播提及“神经生物学著名顶刊 S10 几年前的一项研究”，原音 ASR 字幕作“S10”。该刊号疑为 *Cell* 或 *Science* 量级期刊（讲者口误，未给出确切刊名）；本文按“神经生物学顶刊”含糊处理，不强行指定。

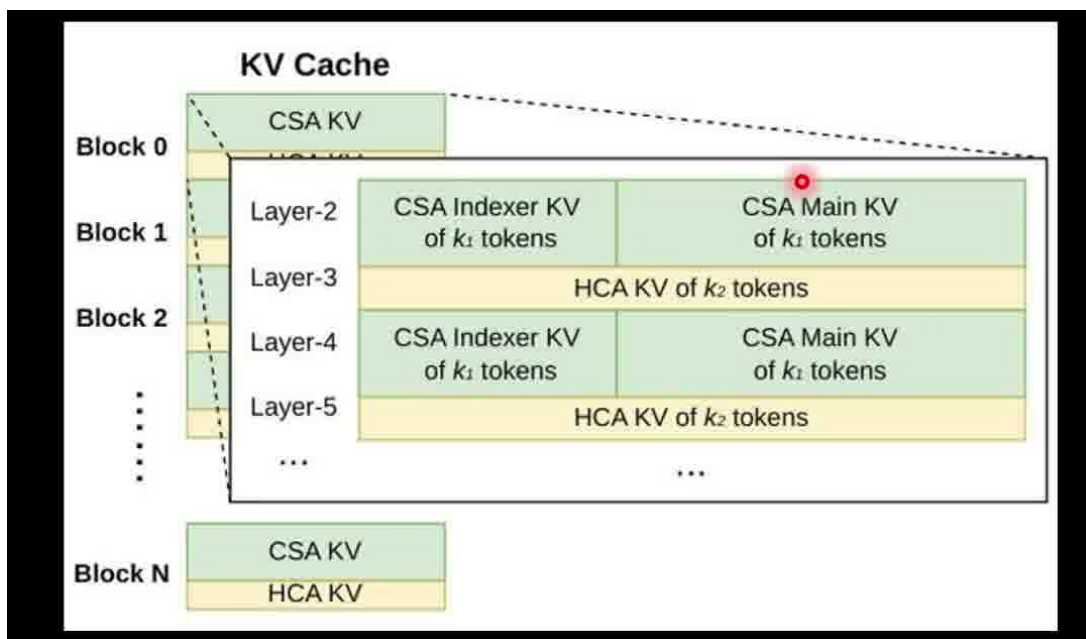


图 7: CSA / HCA 在 layer 维度上的均匀交错：白天觉醒记细节，夜晚睡眠提纯主干⁸

3.3 排除法：四种“看起来也行”的方案为什么都不行

均匀交错并不是“灵机一动”的结果，而是讲者通过一连串排除法逐个否决其它方案后留下的“唯一解”。把这些被否决的方案罗列出来，本身就是一份不错的设计教程（图 8）。

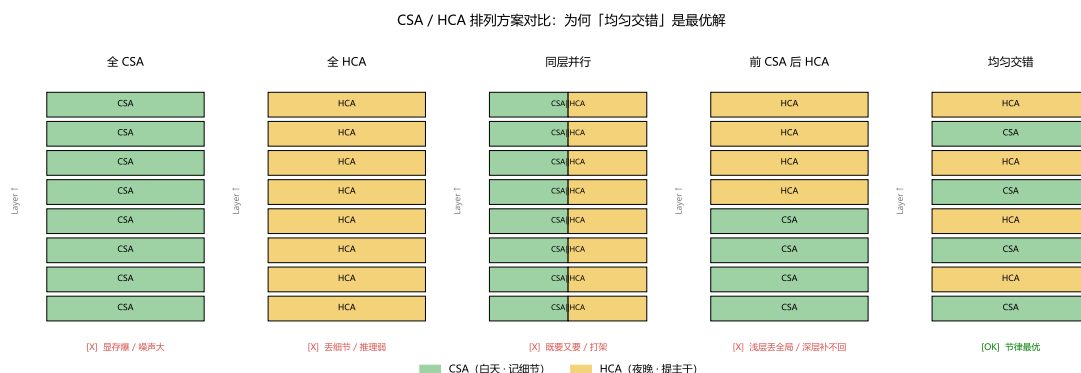


图 8: 五种排列方案的对比：全 CSA / 全 HCA / 同层并行 / 前 CSA 后 HCA / 均匀交错——只有最后一种同时满足“百万 token 上下文 / 训练稳定性 / 算力平衡”三个目标。

1. **全用 CSA**。CSA 的压缩倍率不够，到了百万级 token 上下文，KV 显存先扛不住；同时大量细节噪声会让模型训练更不稳。这就像大脑一直清醒、玩命塞东西，迟早炸裂。
2. **全用 HCA**。显存压力确实降下来了，计算也更快；但细节被高强度压缩抹掉，模型无法支撑复杂推理。相当于人一直处于昏睡状态——记是记住了，但记的全是粗轮廓。
3. **同层并行（CSA 与 HCA 在同一层并联）**。听起来很“既要又要”，但这是一种自相矛盾の設定：你没办法既醒着又睡着。两路互相打架，特征空间被来回拉扯，反而比单路更糟。

⁸ 视频画面时间区间：00:05:43–00:07:00。

4. 前 CSA 后 HCA（浅层全细节、深层全压缩）。浅层全是精细局部信号，会丢失全局；深层全是粗粒度信号，又补不回细节。讲者打了个生动的比方：骆驼也不行——总不能连续好几天不睡觉再使劲补一觉。

只有第五种——**layer-wise 均匀交错**——能同时兼顾以下三个目标：

- 支撑百万 token 上下文（HCA 兜显存）；
- 维持训练稳定性（HCA 持续过滤噪声、CSA 持续保留细节）；
- 取得算力平衡（重活轻活轮流来，没有哪一段特别重）。

3.4 机制：把节律设计写进 Transformer

为什么说均匀交错是“最大程度地模拟人脑节律”？这里有一个值得细品的对应关系：

- Transformer 的层（**layer**）维度对应人脑的时间维度：每过一层等价于“过了一段时间”；
- CSA 层 = 一段觉醒期：保留滑窗短期记忆、动态检索关键 KV、记下当下细节；
- HCA 层 = 一段睡眠期：用超高倍压缩去噪声、剪冗余，把主干从一堆细节里“提纯”出来；
- 一个 CSA-HCA-CSA-HCA-... 序列 = 一连串“白天-夜晚-白天-夜晚”的微循环：模型每“醒”几层、每“睡”几层，反复在记忆与遗忘之间切换。

这种结构的妙处在于：它不是把“记”和“忘”拆成两个阶段做完（那是前 CSA 后 HCA 的失败方案），也不是同时做（那是同层并行的打架方案），而是在同一条 *pipeline* 上交替进行——和我们每天的作息节律完全同构。讲者用一句话点题：干一会儿睡一会儿，越学越稳。

[

关键收束：均匀交错 = 节律对齐] V4 把 CSA 与 HCA 在 Transformer 的 layer 维度上做 **layer-wise 均匀交错**，是一个同时回应百万 token 上下文 / 训练稳定性 / 算力平衡三大目标的构造性最优解；其底层灵感来自神经生物学对节律（*circadian rhythm / sleep*）的研究——清醒编码细节、睡眠固化主干。CSA + HCA 的轮替，等价于把人脑的睡眠—觉醒节律写进了网络结构本身。

3.5 本章小结

本节把 §1 的“主动记忆”和 §2 的“主动遗忘”装进了同一条网络。要点回顾如下。

- **动机**。CSA 和 HCA 都不可或缺，但简单地全用其一、串行或并行都各自失败：显存爆、丢细节、相互打架、前后段补不回——都是非此即彼的极端。
- **机制**。只有 layer-wise 均匀交错才能在同一条 pipeline 上同时维持“记”和“忘”，对应人脑清醒—睡眠节律。
- **结果**。模型获得了一种节律式的训练动力学：每隔几层就压一次冗余、提一次主干，长期来看比单一机制更稳、更省、更耐扩展。
- **下一步**。有了这套记忆—遗忘—节律三件套，就剩下一个问题没回答：怎样把它装回 V4 的整张架构图？这就是 §4 的任务——**内生 RAG**。

4 内生 RAG：把检索焊进 Attention（含 MoE 与 MHC 工程优化）

我们已经有了 CSA + HCA 这套记忆—遗忘机制，下一步是把它装回 V4 整图。本节聚焦讲者的第四个要点：当这套节律化的记忆机制被嵌进 attention 层之后，V4 的整张计算图究竟变成了什么样？又意味着什么样的范式转变？

[

全局坐标：我们在大纲的哪一站] 回顾全局五要点闭环——记忆（CSA）→ 遗忘（HCA）→ 节律（交错）→ 内生 RAG → 内生 Agentic。前三点解释了 V4 attention 子层”长成什么样、为什么这样长”；本节是衔接段：把它装回 Transformer，并顺带回答 MoE / MHC 是怎么配合的。读完本节，读者应该能用观测层 / 传播层 / 决策层三个词概括 V4 全局改动幅度。

4.1 V4 attention block：双路计算单元，不是旁路

V4 的整张架构图本质上仍是一条 attention pipeline，但讲者特别提醒要看清楚一个细节：图中那个看似”挂在主干旁边”的 CSA + HCA 双路块并不是旁路。它就是这条 pipeline 的主体计算单元（block）本身。

把单个 attention block 摊开来看，流程其实非常清晰（图 9）：

1. 块前混合（pre-block mixing）：把进入 block 的特征做一次拆分，分别送往两条路；
2. 双路计算：一路是 CSA，专门”抓细节、做动态检索”；另一路是 HCA，专门”提纯主干、做主动遗忘”；
3. 块后混合（post-block mixing）：把两路的输出融合，再送回主干（residual）。

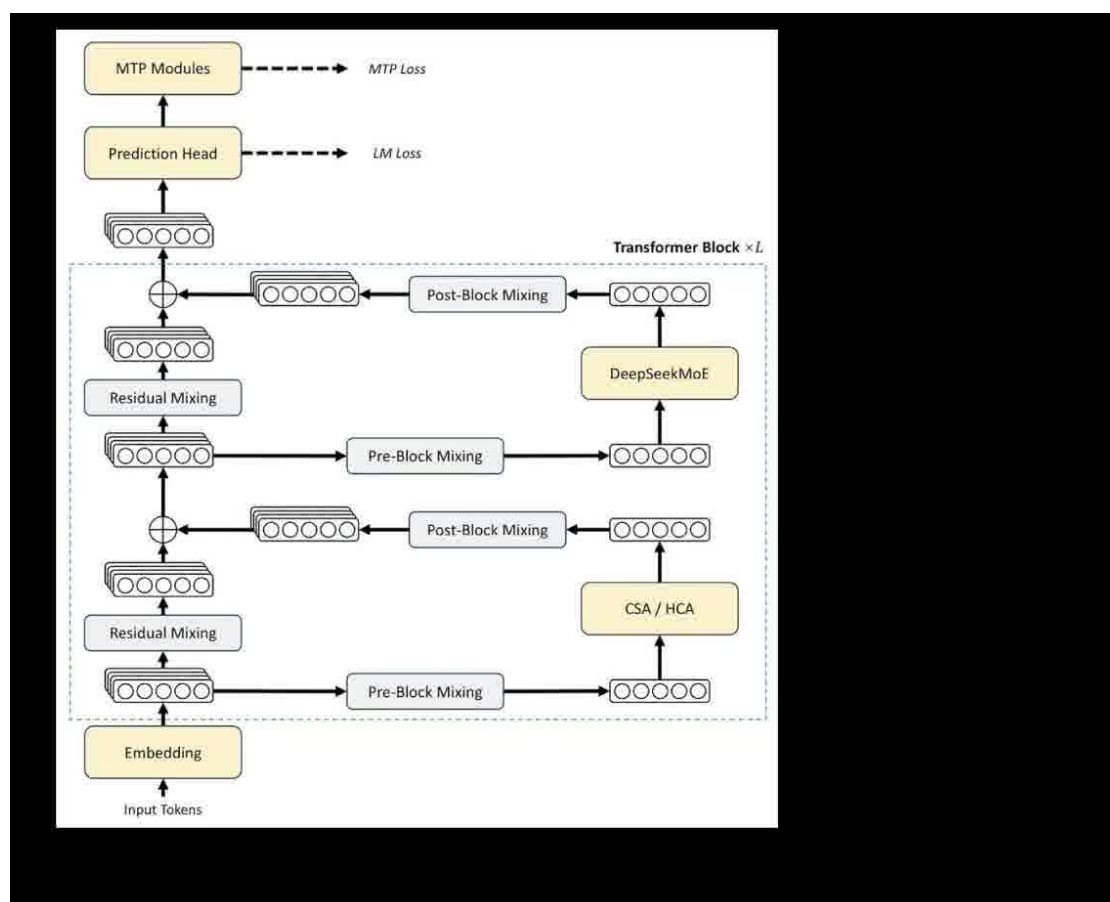


图 9: V4 单个 attention block: pre-block mixing 拆分 → CSA / HCA 双路并行 → post-block mixing 融合 → 回主干。这是 V4 attention 子层的主体计算单元，不是旁路⁹

[

常见误读：双路 = 旁路？] 许多读者第一眼会把 CSA / HCA 的双路结构理解为“挂在 attention 主干旁边的两个辅助分支”。讲者明确纠正了这个误读：这两路就是 **attention** 的主体——pre-block mixing 把特征送进去、post-block mixing 把结果接出来，主干本身就是由这两条路撑起来的。把它当旁路看，会低估 V4 改动的规模。

4.2 从外挂 RAG 到原生检索增强

把 attention block 拆开来看清楚后，再退一步问一个更宏观的问题：这个结构像什么？

它特别像一种把 **RAG** 揉进 **attention** 层的设计。CSA 自带 indexer / index queries / index scores / top-k 召回，这些组件在传统 RAG 体系里对应的就是向量库 + 检索器。区别在于——传统 RAG 是把检索器外挂 in LLM 旁边、通过 prompt 拼接；V4 是把“压缩 + 向量检索”直接焊进了注意力层，做成了端到端的原生检索增强（**native RAG**）（图 10）。

⁹视频画面时间区间：00:08:46–00:09:55。

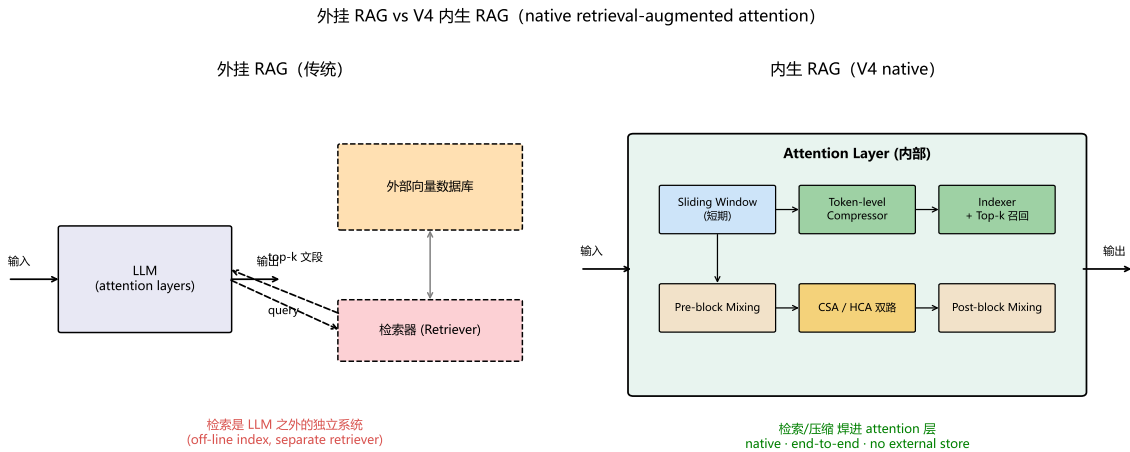


图 10: 外挂 RAG (左) vs. V4 内生 RAG (右): 左侧 LLM 与外部向量库通过检索器拼接成两段式系统; 右侧 attention layer 内部直接含 compressor + indexer + top-k 召回, 端到端可微、无外部存储。

讲者用了一句很有画面感的话来概括这个变化——”把检索能力吃回到自己的体内”。差异并不仅是”快不快”, 而是范式的差异:

- **速度**: 不再需要每次推理走一次外部向量库, 省掉一次 I/O;
- **无缝**: 检索分布与 attention 分布共训练、共优化, 不存在”检索器训得不好拖累生成”的问题;
- **端到端**: 整条计算图都是可微的, 从 token 到检索结果再到输出, 反向传播一以贯之。

[

关键收束: 内生 RAG 是范式之差] V4 attention block 不是”在 Transformer 旁边接了一个 RAG 模块”, 而是把 **RAG** 的检索机制直接焊进 **attention** 层, 形成原生检索增强 (**native RAG / native retrieval-augmented attention**)。这是大模型正在以不可阻挡之势把外部检索能力吃回体内的标志性一步: 从”模型 + 外挂检索”走向”模型 = 模型 + 检索”。

4.3 MoE: 决策层只做工程优化

把目光从 attention 子层挪开, 往上一层就是 MoE 决策层 (FFN)。和 attention 子层的大改相比, MoE 这边的变化要克制得多——讲者反复强调: ”V4 总体上保留了 V3 的 MoE 结构, 只是在工程层面做了一些小优化。”具体改动有三处 (图 11):

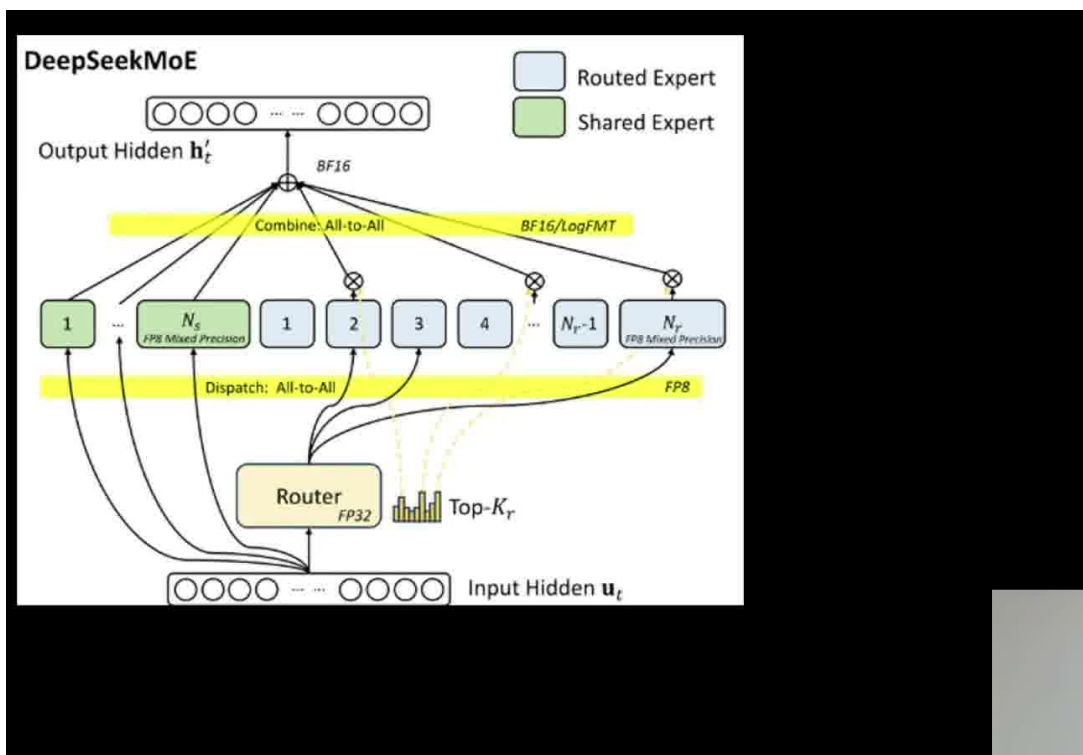


图 11: V4 MoE router 的三处工程微调: sigmoid $\rightarrow$ softplus 平方根; 移除 top-k 专家硬限制; 新增序列级负载均衡损失¹⁰

改动一: 路由门控函数 **sigmoid** $\rightarrow$ **softplus** 平方根。 router 在拿到 token 的隐状态 u_t 后, 要给每个专家打分。V4 把 V3 用的 sigmoid 打分换成了 softplus 的平方根, 可以示意性地写成:

$$\text{score}_i = \sqrt{\text{softplus}(W_i u_t)}, \quad \text{softplus}(x) = \log(1 + e^x).$$

其中 u_t 是 token 在该层的隐状态, W_i 是第 i 个专家对应的路由权重。 $\sqrt{\text{softplus}(\cdot)}$ 与 sigmoid 的差别在于: sigmoid 上界饱和 ($\rightarrow 1$), 平方根 softplus 在大输入区仍然缓慢上升, 能为高分专家保留更细的相对差异, 避免打分被挤到同一段而互相分不开。

改动二: 移除”目标专家数量”硬限制。 V3 的路由会硬性约束每 token 必须激活恰好 top-k 个专家¹¹; V4 放松了这个约束, 让每个 token 自适应地选择一组合适的专家, 提升路由自由度。

改动三: 新增/调整序列级负载均衡损失。 在原有的负载均衡正则项之上, V4 在序列级(sequence-level)加了一项新的均衡损失, 避免某些专家在长上下文下被反复偏爱, 进一步稳住训练。

讲者特意补了一句: 读 V4 技术报告时, 不必在这些工程细节上太抠。它们是”匠人式的打磨”, 不是改变范式的大动作。

¹⁰ 视频画面时间区间: 00:10:10–00:11:00。

¹¹ 具体激活专家数量的数值视频未给出, 详见 DeepSeek V4 技术报告。

4.4 MHC：传播层的智能阀门

最后看一眼模块之间的连接——也就是**传播层**。V4 在不同 block 之间用的是 **MHC**（流形约束超链接，Manifold-Constrained Hyperlink）¹²。从效果上讲，它做了两件事（图 12）：

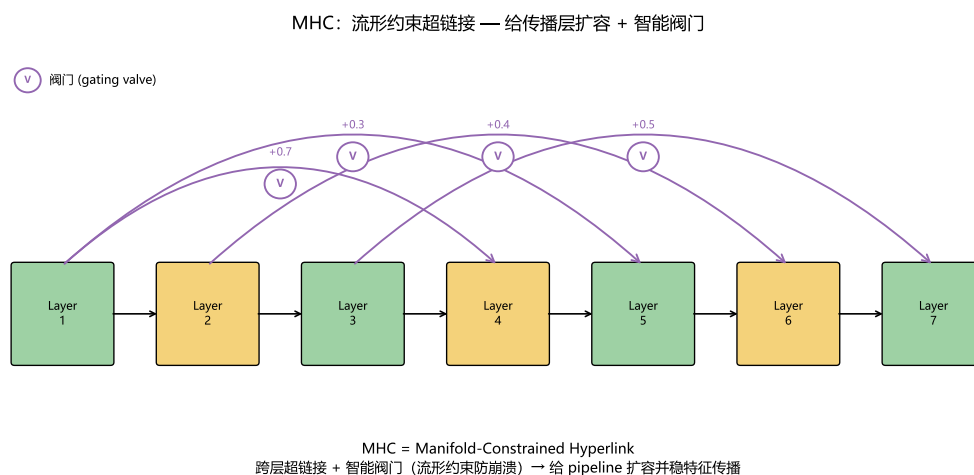


图 12: MHC（流形约束超链接）：在层与层之间架设带”流形约束”的跨层连接，并为每条连接装上”智能阀门”控制信号强度，使训练过程不易崩溃。

- **给 pipeline 扩容**：把原本只能沿主干逐层传播的特征流，扩展为带跨层超链接的多通道结构；
- **加装”智能阀门”**：每条超链接都有可学习的阀门，控制信号强度——这是讲者的原话——确保训练过程中不出现特征塌缩或梯度爆炸。

底层数学动机是流形约束：让特征在跨层传播时仍处于一个低维稳定流形上，避免被链路放大成异常激活。读者只需记住 MHC 的工程定位即可：**它是传播层的”安全总线”**。

[

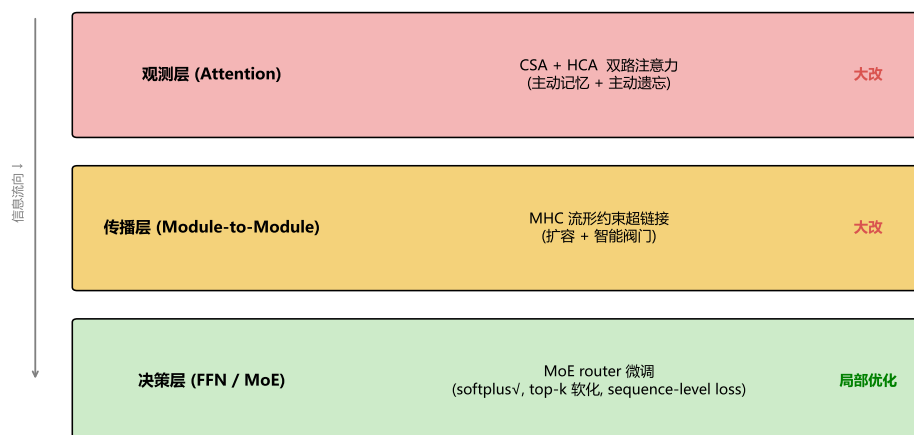
术语提示：MHC] ”MHC / 流形约束超链接 / Manifold-Constrained Hyperlink” 在原视频中由讲者提及，且口语中存在 ASR 误读（如”流行约束”→”流形约束”）。该术语带有较强的讲者—团队个人语境，并非业界通用名词；正文中保持讲者用法，读者在外部检索时建议结合 DeepSeek 公开技术报告对照。

4.5 三层归纳：V4 改动的总体地图

把 §1 - §4 的所有改动放在一张图上，讲者给出了一个非常凝练的归纳框架：把 V4 的整张架构按功能切成三层（图 13），并按改动幅度作排序。

¹²讲者口播中提到这是 DeepSeek 前期工作中已熟知的一个组件；MHC 这一术语带有较强的讲者个人/团队色彩，并非业界统一命名，请读者结合 DeepSeek 技术报告对照理解。

V4 改动的三层归纳：观测层（大改）/ 传播层（大改）/ 决策层（工程优化）



注：此三分法为讲者本人提出的归纳框架，非业界通用。

图 13: V4 改动的三层归纳：观测层（attention）大改 = CSA + HCA；传播层 = MHC 大改；决策层（FFN/MoE）= 仅工程优化¹³¹⁴

三层各自的定位与改动幅度如下：

- **观测层（attention，大改）**：从 V3 的 MLA 替换为 CSA + HCA 的节律化双路结构——这是 V4 真正的大动作，本质上是用记忆机制重构了 attention。
- **传播层（MHC，大改）**：用流形约束超链接给整条 pipeline 扩容，并加装“智能阀门”稳住训练。
- **决策层（FFN / MoE，工程优化）**：保留 V3 的 MoE 主干，仅做 router 函数、top-k 限制、序列级负载均衡三处工程微调。

[

关键收束：三层归纳——V4 的“地图”] V4 真正的大改集中在观测层（CSA + HCA）和传播层（MHC）；决策层（MoE）只做工程打磨。换句话说：V4 改的是“看见了什么、信号怎么流”，没大改“看到之后怎么决策”。这与讲者反复使用的“极客 / 匠人 / 长期主义”基调一致——把记忆机制从外挂检索吃回体内，是一项结构性的长期主义工作，而不是一次刷分。这条三分线，也将成为 §6 总结的概念脊梁。

4.6 本章小结

本节把 §1-§3 的记忆—遗忘—节律机制装回了 V4 整图，并给出了一个总体地图。要点如下。

- **V4 attention block 的形态**：pre-block mixing → CSA / HCA 双路 → post-block mixing → 回主干。这是 attention 子层的主体计算单元，不是旁路。

¹⁴对应讲者口播时间区间：00:11:30-00:12:06；本图为按讲者归纳重绘，非视频原帧。

¹⁴“观测层 / 传播层 / 决策层”是讲者本人在本视频中提出的归纳框架，并非业界通用术语；读者在引用时建议注明来源。

- 内生 RAG (native RAG) 是这一形态的概念名：把”压缩 + 向量检索”焊进 attention 层，与外挂 RAG 形成范式差。讲者一句”把检索能力吃回到自己的体内”是对这一趋势最简洁的注脚。
- MoE 仅做工程优化：sigmoid $\rightarrow$ softplus 平方根、移除 top-k 硬限制、新增序列级负载均衡损失。读 V4 技术报告时不必在这里抠太细。
- MHC (流形约束超链接) 给传播层扩容并加装”智能阀门”，让特征跨层传输保持稳定。
- 三层归纳 (观测 / 传播 / 决策) 把 V4 改动幅度排成”大改 / 大改 / 工程优化”。这条线是 §6 全篇综合的基础。
- 下一步。有了这套内生检索能力和重塑后的 attention 主体，自然会问：把它们当作底座，能否进一步把 Agent 的能力也”内生化”？这就是 §5 的主题——内生 Agent 化。

5 内生 Agent 化：把 Agent 做进模型里

§4 我们看到 DeepSeek V4 把检索焊进 attention，这只是冰山的水面部分。再往上一层看，水面之上的轮廓便是**内生 Agent 化**（native agent）。如果说 §1–§3 的「机制三连」是为模型重塑了记忆器官，§4 的内生 RAG 是把检索这条神经焊进了观测层，那么 §5 要回答的便是另一个问题：当模型已经长出了长程记忆与多轮自循环推理之后，它还需要外面那一圈 *harness* 吗？

5.1 为什么 DeepSeek 一年半不做 Agent？

最近一年半，几乎所有大模型厂商，无论是 OpenAI、Anthropic 还是国内的头部玩家，都在向 Agent 进军：AI coding、long-horizon agent、多文档分析、复杂代码任务和测试时扩展（test-time scaling）几乎成了行业的统一动作。DeepSeek 却在这一年半里几乎一言不发，外界替它着急——「他为什么不干？他到底在干什么？」

讲者借用了一句调侃：「千呼万唤始出来，犹抱琵琶半遮面。」V4 出来后，真正读完技术报告的人才看清梁文峰为什么不动¹⁵：他不是不做 Agent，而是要把 Agent 做进模型里，而不是做在模型外面。这是一个战略级别的判断——当大多数玩家把精力花在 LLM 外部的 harness、workflow、记忆增删调度上时，DeepSeek 选择直接改造 Transformer 的内部结构，让模型本身具备 Agent 的核心能力。

5.2 老式 input-output（图 B）vs V4 内部自循环（图 A）

讲者在白板上画了一组对照图，是理解「内生 Agent 化」最直接的入口。

¹⁵讲者原话：「梁文峰看成了 Agent 的死穴。」此处保留作为强力 takeaway。

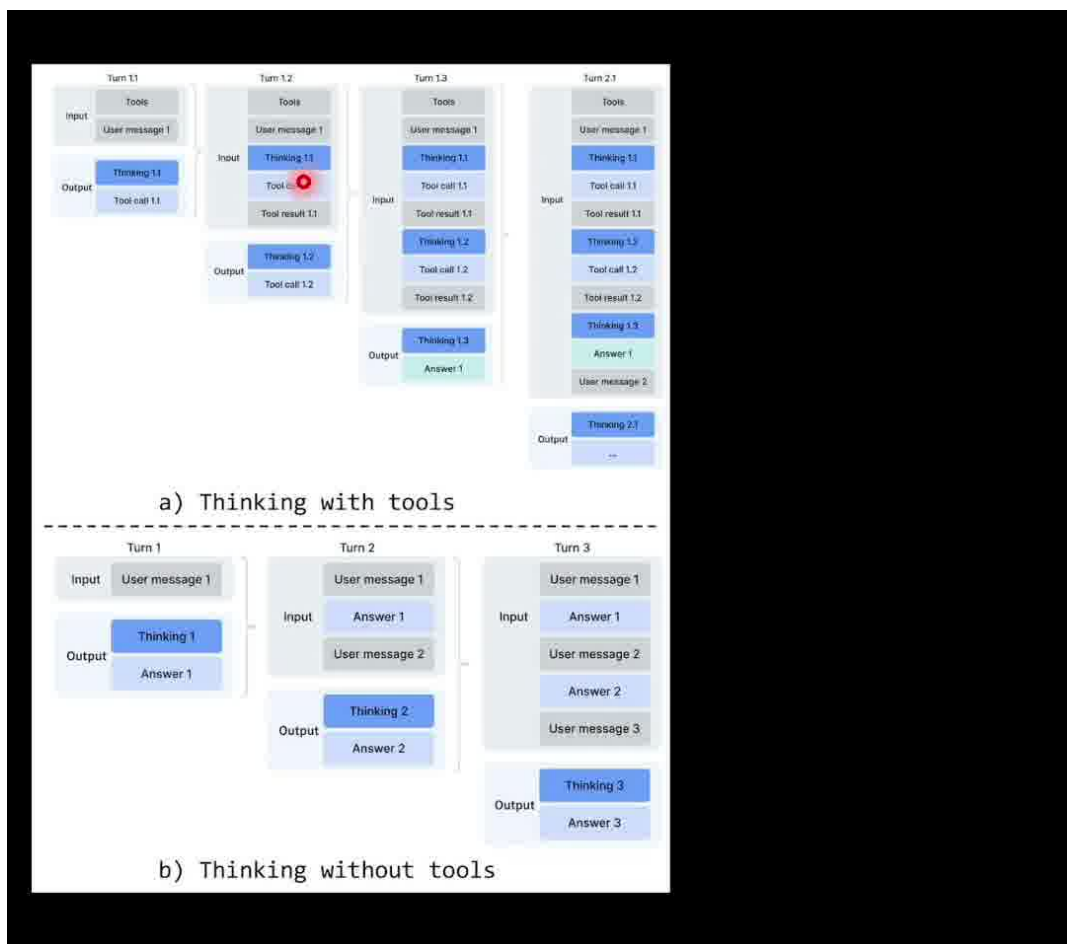


图 14: 老式 input-output 推理（图 B）vs V4 内部多轮自循环 reasoning（图 A）。图 B 是单次问答、无逻辑嵌套的严格 I/O，必须在 LLM 外面套一层 harness 才能形成循环；图 A 则把循环嵌套做到了模型内部。¹⁶

图 B: 老式推理 + 外挂 harness。 用户问一次，模型动一次；模型自己不会主动干活，是一种严格的 input-output。要想得到逻辑嵌套（先思考、再调工具、再思考、再调工具……），就必须在 LLM 外面再套一圈循环。这一圈外挂壳的工程叫做 harness¹⁷/workflow，从早期的 Manus¹⁸到当下大量 Agent 框架，本质上都是这条路线：**LLM 是被动核心，循环逻辑在外部**。

图 A: V4 内部树状自循环。 一次提问之后，模型在内部展开像树一样的多轮自循环：thinking → tool-call → re-thinking → verify → ... → 最终对外输出。换句话说，**长程记忆、自主推理、迭代复盘**这三件 Agent 的核心能力，被通通做进了模型底层。讲者形容这是「在 LLM 内部某种程度上实现了 Agent 化」。

5.3 Request 级别的调用模型：SWA + KV state

把视角从单次推理拉远一格，看 V4 在多轮交互里的工作流。讲者画的第二张图描述了 request 级别的调用模型。

¹⁶视频画面时间区间：00:12:30-00:14:00。源帧若不清，可使用备份重绘版 [figures/fig-agent-loop-vs-internal-redrawn.pdf](#)。

¹⁷讲者口述「在外边加上 isn't 形成一个循环」，「isn't」系 ASR 对 harness 的误识，本笔记统一更正。

¹⁸讲者口述「从 minus 到最近的这些 agent」，「minus」疑为国产 Agent 框架 Manus 的 ASR 误识，标注待核。

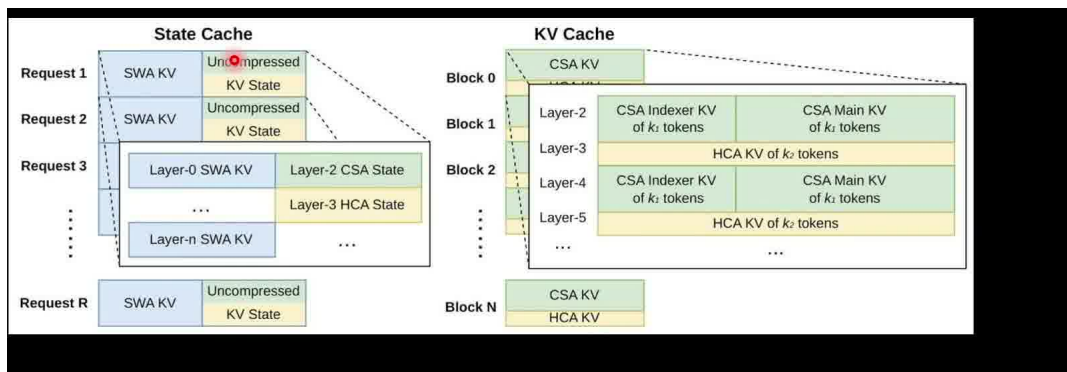


图 15: Request 级调用流：每个 request = 蓝色 SWA（短期、不压缩、几千 token）+ 黄绿色 KV state（长期、压缩、用户级长期记忆库）。短期与长期一起送入计算单元，再分别刷新短期、更新长期。¹⁹

Request 之间相互独立。 每一个用户对话视为一个 request；request 与 request 之间是独立的。

每个 request 的两段记忆。 一个 request 由两部分共同组成：

- **蓝色 SWA**（即 §1 介绍的 Sliding Window Attention，滑动窗口注意力）：短期记忆，不压缩，token 数比较少，量级在「几千」²⁰。
- **黄绿色 KV state**（KV cache 长期态）：用户的长期记忆库，压缩存储的 KV 键值对，量级远大于 SWA。

读写循环。 每次实际调用时：

1. 把短期 SWA + 长期 KV state 一起送入右侧的计算结构，跑一遍前向；
2. 把计算结果翻回左侧，刷新短期 SWA；
3. 同时更新用户级长期 KV state。

读到这里，凡是写过 Agent 的同学应该会有有一种熟悉感——这与 Agent 框架里那个 `memory.md` 文件几乎是同一个范式：短期上下文 + 持久化长期记忆，每轮交互后双向同步。

[

V4 KV state 与 `memory.md` 的对位]

- **相似处。**都把短期上下文与长期记忆显式分离，每轮交互后短期与长期同步刷新；
- **差异处。** V4 的长期 KV state 不能手动编辑、没有可读的语义结构（全是 KV 键值对张量）、没有显式的读写逻辑（读写嵌进 attention 计算里）；
- **关键定性。** V4 KV state 是 内生的、模型层面的长期记忆；`memory.md` 是 外挂的、文件层面的长期记忆；两者共享思想，但所在的层级完全不同。

¹⁹ 视频画面时间区间：00:14:30-00:16:17。备份重绘版 `figures/fig-kv-state-flow-redrawn.pdf`。

²⁰ 视频未给出具体数值；具体数值见原始 DeepSeek V4 技术报告。

5.4 从外挂记忆到内生记忆：梁文峰看到的「Agent 死穴」

把图 A、图 B 与 request 级调用流一起摆在桌面上，DeepSeek 这一年半的沉默就有了答案。

[

战略洞察：内生记忆是 Agent 的基础设施] 当大多数人还在 LLM 外部争论「如何增删记忆 / 如何设计 memory 调度策略 / 如何让 harness 更聪明」的时候，DeepSeek 已经把这件事做进了模型内部。这是从「外挂记忆」到「内生记忆」的关键一步——也是讲者所说的、梁文峰看到的「Agent 的死穴」：上层 harness 工程做得再精巧，只要底层模型不具备长程内生记忆与自循环推理，整个 Agent 体系就永远站在沙地上。V4 把地基重浇了一层。

5.5 重要边界：内生 Agent 化 $\neq$ 完整 Agent

不过，必须把边界讲清楚，否则容易把一个雏形误读成终态。

[

V4 的「内生 Agent 化」目前还不是完整 Agent]

- V4 尚未对接完整的工具调用接口 (tool use)；
- V4 尚未对接完整的执行环境 (沙箱 / 浏览器 / 文件系统等)；
- V4 当前实现的是「原生深度思考的内循环架构」，是把 Agent 的记忆与推理骨架内化到模型里，而不是把整套 Agent 系统都内化掉。
- 因此应把 V4 视为「内生 Agent 化」的雏形，而非终态。完整 Agent 仍需上层 harness 提供工具、环境与编排——只是 harness 的重量会因为模型底层的进化而显著变轻。

5.6 本章小结

- DeepSeek 一年半不做 Agent，不是慢，而是走的层级不同——把 Agent 做进模型，而不是做在模型外。
- 图 B (老式 I/O + harness 外挂) vs 图 A (内部树状自循环) 揭示了两种 Agent 范式的根本差异：循环逻辑在壳外还是壳内。
- Request 级调用模型 = SWA (短期，不压缩，几千 token) + KV state (长期，压缩，用户级记忆库)；每轮双向同步，与 memory.md 范式同构而内生化。
- 战略落点：从外挂记忆到内生记忆，是「Agent 死穴」的破解口。
- 边界落点：V4 是内生 Agent 化的雏形，没有 tool use、没有完整执行环境；它改写了基础设施，而不是替代了 Agent 工程整体。

下一章我们把五要点串成闭环，并讨论这条路线对 Agent 行业、router 架构、后训练范式的连锁影响。

6 总结与延伸

把前五章合上看，DeepSeek V4 给我们的，不是又一组刷榜数字，而是一条 从神经生物学回到 Transformer，再外推到 Agent 形态的连贯叙事。这一章先用「五要点闭环」把全篇主线收紧，再沿着讲者收尾的趋势讨论铺开五条延伸支线，最后由笔者给出一段独立的合成与开放问题。

6.1 五要点闭环

讲者本人用「五个教学要点」组织全篇；闭起来之后，五点之间不是并列，而是 首尾相接的环。

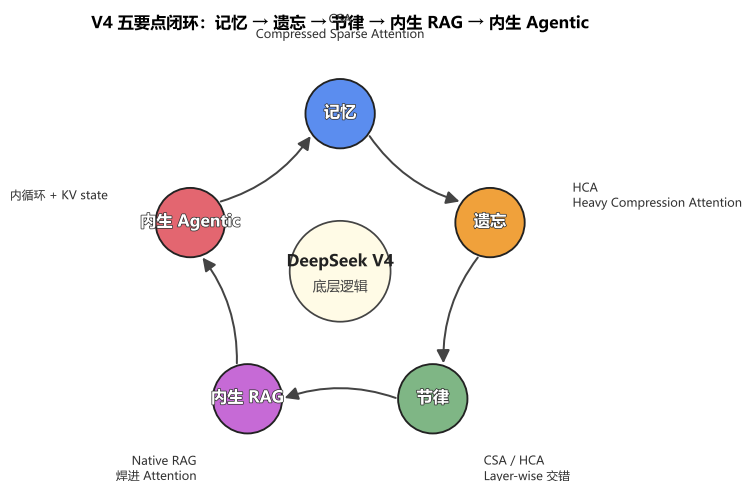


图 16: V4 五要点闭环：记忆（CSA）→ 遗忘（HCA）→ 节律（CSA/HCA 交错）→ 内生 RAG（焊进 attention）→ 内生 Agent 化（内循环 + KV state）→ 回到记忆。²¹

- **1. 记忆（CSA）。**用 sliding window + token-level compressor + indexer + top-k multi-query attention 形成 多分辨率动态长短期记忆；哲学上等价于 LSTM 但不引入额外门控。
- **2. 遗忘（HCA）。**砍掉 query 动态检索，对 KV 做超高倍压缩，过滤细节、提纯主干，对应「主动遗忘巩固记忆」。
- **3. 节律（CSA/HCA 交错）。**layer 维度均匀交错，模拟睡眠 - 觉醒的生物节律，平衡显存、稳定性与细节保留。
- **4. 内生 RAG。**pre-block mixing → CSA/HCA 双路 → post-block mixing，把检索焊进 attention，是原生检索增强而非外挂 RAG。
- **5. 内生 Agent 化。**内部树状自循环 + request 级 SWA / KV state 双段记忆，把 Agent 的核心骨架做进模型底层。

²¹对应视频画面时间区间：00:16:17–00:17:00，讲者口头复述五点时。

[

全篇主线收紧] 五点不是并列的清单，而是一条因果链：记忆为什么要分长短期，于是有 CSA；长期记忆为什么会拖累而非帮助推理，于是有主动遗忘 HCA；二者为什么不能并行不能串行，于是有节律交错；这套机制怎么塞回 Transformer，于是是观测层的内生 RAG；模型自己长出了长程记忆与多轮自循环，于是有了内生 Agent 化——再回头，长期 KV state 又喂养下一轮的记忆/遗忘机制，闭环成立。

6.2 三层归纳再确认

§4 末尾我们提出了讲者的三层归纳框架²²。在 §6 收尾时，讲者再次主动把它拉回来：

- 观测层 (attention) = 大改 = CSA + HCA。严格意义上是一种检索增强；讲者称之为「记忆」，并指出记忆 / 遗忘 / 节律三者共同使长短期记忆变得动态、交互性、精准性、稳定性、可训性、可迁移性都更强。
- 传播层 (modules between blocks) = 大改 = MHC (流形约束超链接)。拼命扩容、增加「智能阀门」，用流形约束让特征传递更稳，整体训练结构更稳。
- 决策层 (FFN / MoE) = 局部优化。仍是 MoE 架构，主要是工程层面的微调 (softplus 平方根、移除 top-k 硬限制、序列级负载均衡损失)。讲者明确点出这一层「还有很大改进空间」。

这三层一一对位，正是 V4「真正动了哪儿、轻轻碰了哪儿、留待下一代再动哪儿」的清晰索引。

6.3 弱 Agent 危机：哪些 Agent 会消亡，哪些不会

讲者在 §6 中给出一段对从业者颇为残酷但也颇为实在的判断。

「每次模型的大的迭代，都会让很多 Agent 存在的意义大幅降低，很多就消亡了。」

他把当前市场上「纯靠外挂框架、纯做 workflow」的 Agent 称作弱 Agent (弱 Agentic)：这一类 Agent 几乎不引入新的领域知识，只是把现有 LLM 调一调、串一串。模型原生能力一旦快速迭代——比如 V4 把长程记忆和内循环做进了底层——这一层壳的存在意义就会被快速侵蚀。

但讲者并不是「Agent 全盘消亡论」。他明确给出三类仍有不可替代价值的 Agent：

- 强行业适配。与具体行业流程、数据、规范深度耦合的 Agent；
- 强可控。必须做严格审计、留痕、合规、可干预的 Agent；
- 强解释性。决策链路必须可追溯、可质询的 Agent。

[

弱 Agent 危机的实操含义] 纯调库 + workflow 的 Agent 开发处在底层模型迭代的正面冲击带。当模型自己长出了长程记忆、内循环推理、原生检索增强，外挂层做的「记忆增删 + 工具串联 + 简单循环」迟早被吃掉一大半。短期红利 ≠ 长期主义。

²² 「观测层 / 传播层 / 决策层」是讲者本人提出的归纳术语，并非业界通用术语；这里作为读者的认知锚点重述。

6.4 Router 趋势：从手动模式切换到 automatic routing

讲者把决策层「还有改进空间」的话头落到了 **router 机制** 上。当前 V4（以及目前主流大模型）仍要用户在 *fast* / *thinking* / *deep-thinking* 之间手选。这是为了适配用户当下的使用习惯而做的 妥协，并非终态。

讲者认为，未来应当走向**自动路由（automatic routing）**²³：由模型本身根据输入难度、上下文长度、任务类型，自动决定走哪条计算路径——用户无需手选模式，更不必学一堆 prompt 技巧去触发不同档位。

这一步的意义不仅是「用户体验更好」，更是把元控制权收回模型内部，与「内生 Agent 化」是同一个方向上的姊妹趋势。

6.5 后训练 / 行业能力被吃进通用大模型

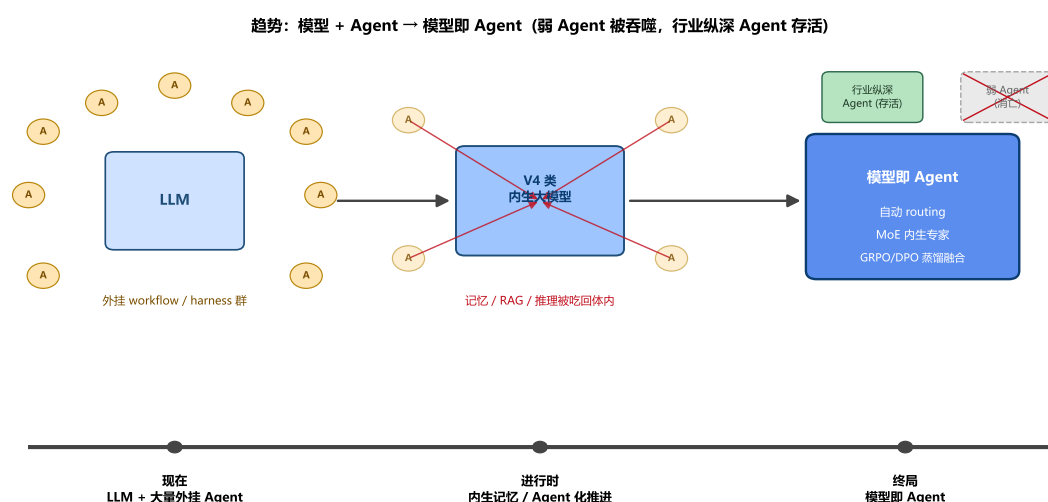


图 17: 趋势示意：从「LLM + 大量外挂 Agent」→「内生记忆 / 内生 Agent 化推进」→「通用大模型 + 自动 routing + 后训练分类融合 + 行业专家被吃进 MoE」。底部标注：弱 Agent 消亡 / 行业纵深 Agent 存活。²⁴

讲者用很短的篇幅一笔带过 V4 的后训练组织模式，但这一笔信息密度很高：

- **多教师策略（multi-teacher）**：多个专家模型联合指导学生；
- **蒸馏（distillation）**：把多教师能力压进单一通用模型；
- **GRPO 类后训练融合**²⁵；
- **先分类再融合**：基于多任务 / 多专家分类 → 再做融合蒸馏。

把这条工艺线和 §4 的 MoE「自动 switch」连起来看，趋势就出来了：

预训练阶段是非常通用的大模型；所有行业能力会被大模型「吃掉」，吃完之后基于混合专家结构自动 *switch* 到对应专家。

²³讲者口述「应该像 GPT-5 之前做的那样、第一版做的那样」，此处对应的是 GPT-5 早期版本引入的 router 自动切换 *fast* / *thinking* 模式的设计；本笔记按其原意写作，记为「GPT-5 引入的自动路由」。

²⁴对应视频画面时间区间：00:17:00–00:21:36（不含 21:36 之后的招生 promo）。

²⁵讲者口述为「OPD 融合」，疑为 *GRPO*（Group Relative Policy Optimization，DeepSeek 系列的 RL 后训练算法）或 *DPO* 的 ASR 误识。本笔记按 GRPO 处理并加此脚注；具体算法名称以 DeepSeek V4 技术报告为准。

这一步推到极致，「行业大模型 / 行业 Agent」这两个曾经被反复讨论的概念，都可能被「通用大模型 + 内生专家 MoE 自动切换」自动替代——前者承载知识广度，后者承载行业纵深，整套架构在模型内部完成而非通过外挂壳串联。讲者用 AI coding 与文生图（如 nano-banana 与主流图像编辑模型²⁶）做例子说明：当一个行业的能力被通用模型整合得足够好，原先围绕该行业的大量外挂 Agent 也将随之 disappear。

6.6 终局判断：从「模型 + Agent」到「模型即 Agent」

把上面四条延伸——内生 Agent 化、router 自动化、后训练吃进行业能力、弱 Agent 危机——叠在一起，讲者给出收尾断言：

「下半年，可能是明年。很可能不再是模型加上 Agent 了，而是模型本身就是 Agent。」

视频开头那句话里的关键字，是「道」。讲者反复强调：DeepSeek 一年半的沉默，不是没活儿干，而是在坚持一种长期主义——不去追每一波短期红利、不在外挂壳上反复造轮子，而是把基础设施一砖一瓦地重浇。V4 让我们看清的不只是几个新的 attention 模块，更是这条路线在工程上是真的可走的。

[

「道」与长期主义] 道 = 把 Agent 化做进模型本身，而非做在模型外；做长程记忆、做节律、做内生检索、做自动路由——所有这些都是同一种长期主义的不同切片。短期看上去 DeepSeek 「慢」，长期看是它把别人需要回头重做的活儿一次性做完了。

6.7 笔者综合：跨章节压缩与对学习者的实操含义

最后一节是笔者基于全篇的合成（不在视频原话之内），目的是把分散在五章里的概念再压缩一次，并落到对学习者的具体的、可操作的判断上。

(A) 跨章节的概念压缩：从机制三连到 Agentic 含义。

- 机制三连 (§1–§3)：CSA / HCA / 节律——记忆器官层的重塑；
- 架构整合 (§4)：内生 RAG + MHC + MoE 微调——把器官装回 Transformer 的整图；
- Agentic 含义 (§5–§6)：内循环 + KV state + 自动路由 + 行业能力内化——把 Agent 的骨架收进模型。

这是一条由内向外的叙事链：从神经生物学的记忆隐喻，一步步外推到 Transformer 子层、再外推到完整 Agent 形态。每一层都是上一层的承接，没有跳跃。

(B) 对学习者的实操含义。讲者在 §6 收尾时明确说过的一句话是：「一定要做一点复杂的事情，做科研级别的项目，一定要找一个行业深长」——本笔记把这一点放大：

- 高风险方向：纯调库、纯 workflow、纯外挂记忆调度的 Agent 项目。这一层会随着每一代基础模型的更新被反复冲击；零零碎碎的外挂在「能力强的人」面前迅速失去价值。

²⁶讲者口述「无论是 nana banana 还是 edit to」。「nana banana」规范化为 nano-banana（Google 系图像生成模型代号）；「edit to」疑为某主流图像编辑模型的 ASR 误识，本笔记含糊化为「主流图像编辑模型」并标注待核。

- **相对稳健的方向：**行业纵深 + 强交互 + 强可控 + 强解释，或者直接做科研级别的、复杂度高的项目。这一类不容易被通用模型一口吃掉，因为它的护城河不在「壳的厚度」而在「问题的难度」。
- **基础研究方向：**跟随 V4 这类底层范式的演化——记忆机制、节律式 attention 排布、内生 RAG、内生 Agent 化、自动路由、后训练分类融合——是风口在模型而不在壳的人最该花时间的地方。

(C) 留两个开放问题。

1. **自动路由能走多远？**当 router 自动化吃掉用户手选的 fast / thinking / deep-thinking 模式之后，它是否会进一步吃掉提示词工程当前承担的「模式触发」职能？这意味着大量基于 prompt 切换调用模式的 Agent 工程实践都会被改写。
2. **harness 工程的存活形态？**内生 Agent 化进一步推进之后，外部 harness 是会萎缩为薄薄的「工具与执行环境适配层」，还是反而被迫上移、负责更高层的多 Agent 协作、安全审计与可解释性？这是一道关于「壳还剩多少」的问题。

6.8 本章小结

- **五要点闭环：**记忆 → 遗忘 → 节律 → 内生 RAG → 内生 Agent 化 → 回到记忆。这是 V4 的底层逻辑，也是本笔记的主线。
- **三层归纳再确认：**观测层大改、传播层大改、决策层局部优化。
- **弱 Agent 危机：**纯外挂 workflow 的 Agent 在底层模型迭代下生存空间收窄；强行业 / 强可控 / 强解释的 Agent 仍是长期主义的安全区。
- **Router 与后训练趋势：**从手动模式切换走向自动路由；从「行业大模型 / 行业 Agent」走向「通用模型 + 内生专家 MoE 自动 switch」。
- **终局判断：**下半年到明年，「模型 + Agent」可能让位于「模型即 Agent」。这就是讲者所说的、DeepSeek 在坚持的「道」与长期主义。
- **对学习者：**远离纯调库 + 纯 workflow，靠近行业纵深 + 科研级别项目，跟随底层范式的演化——这是更稳的长期主义。

读懂 V4，不是读懂几个新 attention 模块，而是读懂为什么它把这些模块拼成今天这个样子；读懂这一点，再读下半年大模型与 Agent 行业的下一步走向，就不必再「替 DeepSeek 着急」。

6.9 拓展阅读

- **DeepSeek V4 技术报告。**CSA / HCA / MHC / MoE 微调的具体超参（如 SWA 长度、压缩比、top-k 数量）以官方技术报告为准；本笔记中所有「几千 token」「数十万 token」类描述均为讲者口述量级，未给出精确数值。